

■ 第二章

TOOLS

工具调用

让 Agent 从「会回答」变成「会行动」—— 手、眼睛和耳朵，都是工具给的。

FUNCTIONS · TOOLKITS · MCP · HOOKS

工具 = Agent 调用函数去碰外部系统

“Tools are **functions** Agents call to interact with **external systems**.”

— docs.agno.com/tools/overview (官方原话)

① Python 函数

任意普通函数直接放进 tools 列表零装饰、零包装

② Toolkit 工具包

官方预置 120+ 协同函数集合搜索、金融、数据库、文件...

③ MCPTools

把任意 MCP server 的工具挂上来标准化的外部接口

一个普通函数，**就是**一个工具

```
def get_current_time() -> str:
    """
    获取当前时间

    Returns:
        str: 当前时间字符串
    """
    from datetime import datetime
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# 注册: 直接放进 tools 列表
agent = Agent(..., tools=[get_current_time])
```

- 不用装饰器、不用继承，函数即工具
- 函数名、docstring、类型注解都会被 Agno 读取
下一页看它们去了哪
- 返回值建议用字符串 / 数字 / 字典
易序列化的结果模型才好消化

03

工具 schema 是自动生成的

你写的	Agno 生成的 JSON schema	模型看到后的作用
函数名	<code>name</code>	选工具时的名字，要有意义
docstring 正文	<code>description</code>	判断「什么时候该用它」
docstring 的 Args: 段	每个参数的 <code>description</code>	判断「该传什么值」
类型注解	参数 <code>type</code> ；无默认值 → <code>required</code>	保证调用格式正确

官方 Tip: 写全 docstring 和 Args: 段 —— 文档质量直接决定调用质量。

工具调用循环 LLM Loop

1 发请求 run context + 全部工具 schema 一起发给模型	2 模型决策 回复正文，或回复一个/多个 tool call	3 Agno 执行 真正运行函数，把结果塞回对话	4 循环 直到模型不再发 tool call，输出最终回答
---	---	---	--

⚠ 关键认知：调不调用，模型说了算

官方措辞是 "when it deems it relevant"（它认为相关时才调）。代码里写了 tools，不代表每次都会用。

问「15 的平方」，模型可能心算完事

现象

明明给了 `calculate_square` 工具，模型有时直接答 225，根本不调。

多步任务也一样：04_more_tool.py 每次运行，中间步骤不保证一致。

对策（按顺序试）

- ① instructions 写死：“必须使用工具计算，不得口算”
- ② 工具描述写清适用场景
- ③ 还不能容忍 → 别靠概率，走 Workflow 固化（ch6）

「必须调用工具」是提示词约束，不是代码约束 —— 要确定性就得换思路。

Toolkit: 一串协同函数的集合

```
tools=[
    DuckDuckGoTools(
        backend="auto",
        enable_news=False,
        fixed_max_results=5,
    )
],
tool_hooks=[handle_search_errors],
```

- 官方定义：一组可加进 Agent、协同工作的函数集合
内置 120+ 个 Toolkit
- 实际注册进 Agent 的工具名是 `web_search`
- 注意：DuckDuckGo 只返回摘要，不读网页全文
来源和结论要自己核对

给一串工具，模型自己排兵布阵

```
tools=[calculate_square, add, multiply], # 绑定工具列表  
  
agent.print_response("计算 (3-6) *5的结果之后在求平方")
```

- 模型自己规划：先减、再乘、最后平方
调用顺序由模型生成，不是你写的流程
- 一次响应可携带多个 tool call
- 同一句输入，两次运行的中间步骤可能不同
这是特性，不是 bug；要稳定流程请等 ch6



tool_hooks: 把异常翻译成模型能懂的话

```
def handle_search_errors(function_call, arguments):  
    try:  
        return function_call(**arguments)  
    except DDGSEException:  
        return "搜索未返回结果。请缩短关键词后再次调用 web_search; "  
        "当前调用没有可引用的信息来源。"
```

- 包在工具调用外面：拦截异常，返回提示字符串而不是炸掉
- 模型读到提示后会自己调整关键词重试
把「怎么办」的决定权留给模型
- 可预期的 I/O 错误 → 返回清晰错误信息，是官方推荐的工具写法



内置参数： Agno 偷偷塞给工具的值

声明这个参数	工具能拿到什么	模型看得到吗
<code>run_context</code>	运行上下文； <code>session_state</code> 可读写跨对话状态	看不到 —— 从 schema 里剥离
<code>agent / team</code>	Agent 实例本身（如读 <code>agent.instructions</code> ）	看不到
<code>images / videos / files...</code>	本轮输入的媒体文件	看不到

它们在执行时由 Agno 注入 —— 模型永远填不了这些参数。



并发工具执行的三个前提

- 用异步入口: `arun` / `aprint_response`

同步 `print_response` 下多次调用是顺序执行的

- 模型支持并行 **function calling**

官方以 OpenAI 的 `parallel_tool_calls` (默认开) 为例

- 同步函数也能并发: Agno 会把它们放到独立线程上跑

一句话

模型在同一响应里请求多个工具 + 你用了异步入口 = 这些工具**同时**执行, 省的是实打实的等待时间。

MCP 工具的三个冷知识

■ 默认 transport 是 stdio (本地子进程), 远程用 `streamable-http` 或 `sse`

■ 推荐显式 `connect()` / `close()` 管理连接

或用 `async with`; AgentOS 里生命周期自动管理

■ 不管连接的代价: 每次 run 都新建/关闭连接并刷新工具列表

官方原话: not recommended for production use

回顾 ch1: 异步连接必须配 `aprint_response`, 别改回同步。



这四颗雷，排掉就是经验

show_tool_calls 不存在了

当前版本 Agent 构造不收这个参数，你的示例已注释掉 —— 别从旧教程里抄回来。

日志写 OpenAI API 是假象

DeepSeek 适配器复用 OpenAI 兼容客户端，报错标题不代表请求去了 OpenAI，先查 key 和网络。

DuckDuckGo 限流

连续请求过快会触发限制，稍后再跑通常比立即狂重试更有效；查询太窄也会 No results。

import 即请求

01-04 示例在模块顶层调模型、顶层校验 key，import 文件就会发请求、烧 token。



检查点四连问

01 工具的 JSON schema 是怎么从你的 Python 函数生成的？

提示：函数名 / docstring / Args: / 类型注解

02 什么时候写自定义函数，什么时候直接用 Toolkit？

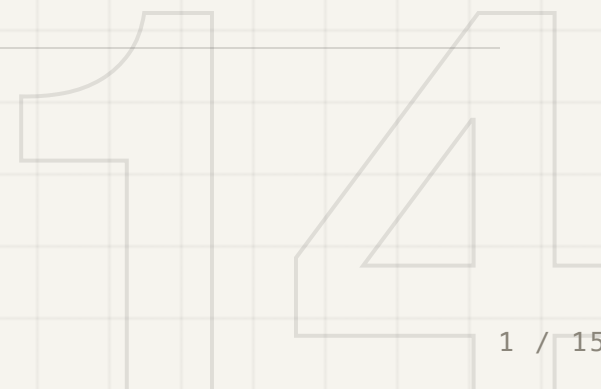
提示：专有逻辑 vs 现成能力

03 写了 tools=[...]，模型保证每次都调吗？怎么提高必调率？

提示：不保证；instructions 写「必须」

04 多个工具调用什么时候会并发执行？

提示：异步入口 + 模型支持并行 function calling



工具 = 函数 + 说明书 模型决定何时用，你决定有什么可用

下一章 · ch3 存储与记忆：让 Agent 记得你是谁。

按 [←](#) 返回复习 · [Home](#) 回到封面